

Project 3:House Price Prediction

importing the Dependencies

```
In [6]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.datasets
from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from sklearn import metrics
```

importing the Boston House Price Dataset

```
In [4]: import pandas as pd
df=pd.read_csv(r"D:\Projects\archive (2)\Housing.csv")
print(df.head())
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished

```
In [8]: print(df.columns)
```

```
Index(['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad',
      'guestroom', 'basement', 'hotwaterheating', 'airconditioning',
      'parking', 'prefarea', 'furnishingstatus'],
      dtype='object')
```

```
In [9]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   price                 545 non-null   int64
1   area                 545 non-null   int64
2   bedrooms             545 non-null   int64
3   bathrooms            545 non-null   int64
4   stories              545 non-null   int64
5   mainroad             545 non-null   object
6   guestroom           545 non-null   object
7   basement            545 non-null   object
8   hotwaterheating     545 non-null   object
9   airconditioning     545 non-null   object
10  parking              545 non-null   int64
11  prefarea             545 non-null   object
12  furnishingstatus    545 non-null   object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
None
```

```
In [17]: from sklearn.datasets import fetch_california_housing
house_price_dataset=fetch_california_housing()
```

```
In [18]: #Loading the dataset to a pandas DataFrame
house_price_dataframe=pd.DataFrame(house_price_dataset.data,columns=house_price_
```

```
In [19]: #print first 5 rows of our DataFrame
house_price_dataframe.head()
```

```
Out[19]:
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longit
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-12
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-12
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-12
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-12
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-12

◀ ————— ▶

```
In [24]: # add the target(price)columns to the DataFrame
house_price_dataframe['price']=house_price_dataset.target
```

```
In [25]: house_price_dataframe.head()
```

Out[25]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longit
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-12
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-12
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-12
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-12
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-12

In [26]: *#Checking the number of rows and columns in the data frame*
 house_price_dataframe.shape

Out[26]: (20640, 9)

In [27]: *#check for missing values*
 house_price_dataframe.isnull().sum()

Out[27]: MedInc 0
 HouseAge 0
 AveRooms 0
 AveBedrms 0
 Population 0
 AveOccup 0
 Latitude 0
 Longitude 0
 price 0
 dtype: int64

In [28]: *#statistical measures of the dataset*
 house_price_dataframe.describe()

Out[28]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOc
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333

Understanding the correlation between various features in the dataset

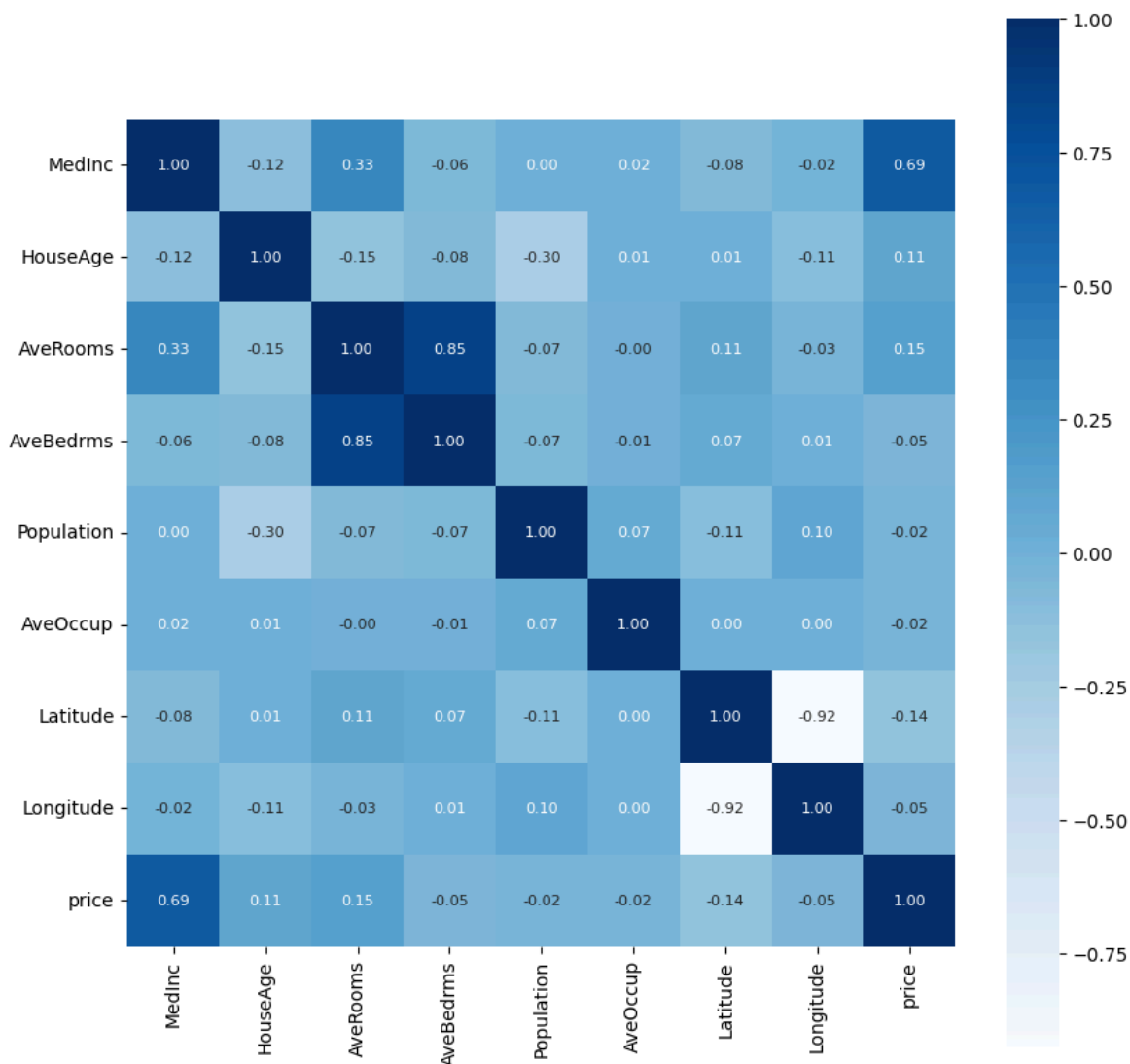
1.Positive Correlation

2.Negative Correlation

```
In [30]: correlation=house_price_dataframe.corr()
```

```
In [37]: #constructing a heatmap to understand the correlation
plt.figure(figsize=(10,10))
sns.heatmap(correlation,cbar=True,square=True,fmt='.2f',annot=True,annot_kws={'s
```

```
Out[37]: <Axes: >
```



Splitting the data and Target

```
In [38]: x=house_price_dataframe.drop(['price'],axis=1)
y=house_price_dataframe['price']
```

```
In [40]: print(x)
print(y)
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	
...	...	...	...	...	...	...	...	
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	

	Longitude
0	-122.23
1	-122.22
2	-122.24
3	-122.25
4	-122.25
...	...
20635	-121.09
20636	-121.21
20637	-121.22
20638	-121.32
20639	-121.24

[20640 rows x 8 columns]

0	4.526
1	3.585
2	3.521
3	3.413
4	3.422
...	...
20635	0.781
20636	0.771
20637	0.923
20638	0.847
20639	0.894

Name: price, Length: 20640, dtype: float64

Splitting the data into Training Data and Test Data

```
In [41]: x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=2)
```

```
In [42]: print(x.shape,x_train.shape,x_test.shape)
```

(20640, 8) (16512, 8) (4128, 8)

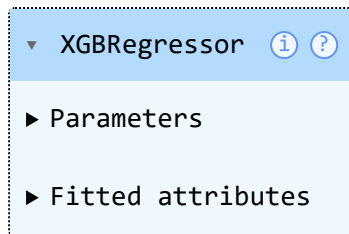
Model Training

XGBoost Regressor

```
In [43]: #loading the model
model=XGBRegressor()
```

```
In [44]: #training the model with x_train
model.fit(x_train,y_train)
```

Out[44]:



Evaluation

Prediction on training data

```
In [45]: #accuracy for prediction on training data
training_data_prediction=model.predict(x_train)
print(training_data_prediction)
```

```
[0.53758794 2.9007688 0.6848911 ... 1.7890452 1.9554564 0.7771621 ]
```

```
In [48]: #R squared error
score_1=metrics.r2_score(y_train,training_data_prediction)
#Mean Absolute Error
score_2=metrics.mean_absolute_error(y_train,training_data_prediction)
print("R squared error:",score_1)
print("Mean absolute Error:",score_2)
```

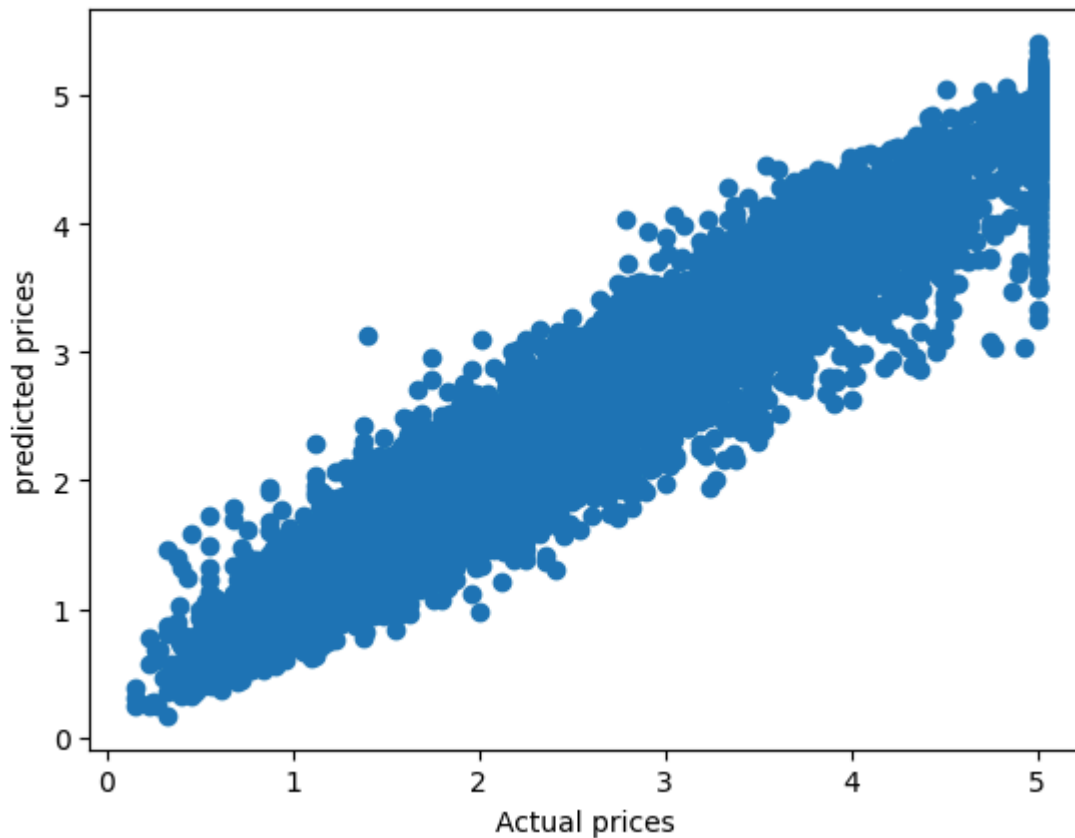
```
R squared error: 0.947186443380359
```

```
Mean absolute Error: 0.18885009247854148
```

Visualizing the actual Prices and predicted price

```
In [50]: plt.scatter(y_train,training_data_prediction)
plt.xlabel("Actual prices")
plt.ylabel("predicted prices")
print("Actual price vs Predicted Price")
plt.show()
```

Actual price vs Predicted Price



Prediction on Test Data

```
In [49]: #accuracy for prediction on training data
test_data_prediction=model.predict(x_test)
print(test_data_prediction)
```

```
[2.880331  1.8918151  0.923897  ... 1.5574043  0.82967275 1.9735563 ]
```

```
In [51]: #R squared error
score_1=metrics.r2_score(y_test,test_data_prediction)

#Mean Absolute Error
score_2=metrics.mean_absolute_error(y_test,test_data_prediction)

print("R squared error:",score_1)
print("Mean absolute Error:",score_2)
```

```
R squared error: 0.8370813165448978
Mean absolute Error: 0.3079223702763229
```

```
In [ ]:
```